would build a Java `Stream` object with `Stream.of()`. The difference is in the reactive behavior.

You can use the `Mono` object to start a reactive stream and then rely on the `flatMap()` operator to pass the data to `OrderRepository`. Add the following code to the `OrderService` class, and complete the business logic implementation.

Listing 8.13 Persisting rejected orders upon submitting order requests

```
...
public Mono<Order> submitOrder(String isbn, int quantity) {
  return Mono.just(buildRejectedOrder(isbn, quantity))        ❶
    .flatMap(orderRepository::save);                          ❷
}

public static Order buildRejectedOrder(
  String bookIsbn, int quantity
) {                                                           ❸
  return Order.of(bookIsbn, null, null, quantity, OrderStatus.REJECTED);
}
...
```

❶ Creates a “Mono” out of an “Order” object

❷ Saves the Order object produced asynchronously by the previous step of the reactive stream into the database

❸ When an order is rejected, we only specify ISBN, quantity, and status. Spring Data takes care of adding identifier, version, and audit metadata.

`map` vs. `flatMap`

When using Reactor, choosing between the `map()` and `flatMap()` operators is usually a source of confusion. Both operators return a reactive stream (either `Mono<T>` or `Flux<T>`), but while `map()` maps between two standard Java types, `flatMap()` maps from a Java type to another reactive stream.

In listing 8.13, we map from an object of type `Order` to a `Mono<Order>` (which is returned by `OrderRepository`). Since the `map()` operator expects the target type not to be a